ESTRUCTURA DE COMPUTADORES

GRADO EN INGENIERÍA INFORMÁTICA

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y ADMINISTRACIÓN DE EMPRESAS

GRADO EN MATEMÁTICAS Y COMPUTACIÓN

uc3m | Universidad Carlos III de Madrid

Grupo de Arquitectura de Computadores

Práctica 1
Introducción al lenguaje ensamblador

Curso 2026/2027

Contenidos

Objetivos de la práctica 3

Enunciado 3

Requisitos a tener en cuenta en la realización de los ejercicios 6

Aspectos importantes a tener en cuenta 7

- **Normas generales 7**
- **Memoria de la práctica 8**

Definición de pruebas 8

Procedimiento de entrega de la práctica. 9

Evaluación de la práctica 10

Objetivos de la práctica

El objetivo de la práctica consiste en entender y aplicar los conceptos relacionados con la programación en ensamblador. Para ello, se utilizará como base el ensamblador del RISC-V (RV32IMF) y el simulador CREATOR al que se puede acceder desde Aula Global

Para familiarizarse con la programación en ensamblador RISC-V y el simulador CREATOR se recomienda resolver los **ejercicios disponibles en Aula Global** en el apartado correspondiente a la primera práctica.

Enunciado

El propósito de la práctica es realizar una serie de funciones en ensamblador RISC-V para la manipulación de sopas de letras.

Una sopa de letras vendrá definida por una matriz de caracteres de F filas y C columnas. Para representar una casilla rellena se utiliza el código ASCII 182 (símbolo ¶). En la siguiente figura se muestra una sopa de letras de 6 filas por 5 columnas:

				¶
	¶			
	¶	¶		

La matriz se almacenará en memoria por filas.

Se asume que la primera coordenada de la matriz es (0,0), es decir, la esquina superior izquierda. De esta forma, en la sopa de letras anterior, los caracteres de relleno se encuentran en las coordenadas (0,4), (1,1), (3,1) y (3,2).

Cuando se quiere reservar una sopa de letras de 10 filas por 6 columnas en ensamblador habrá de definirse en el segmento de datos espacio para representar la matriz correspondiente:

```
.data

   puzzle: .space 60
```

De esta forma se reserva espacio para una matriz de 60 bytes (10 x 6) cuya primera componente (0,0) se encuentra en la dirección `puzzle`.

Se pide desarrollar las siguientes funciones en ensamblador:

`puzzle_print`

Esta función acepta tres argumentos en el siguiente orden:

1. Dirección de la matriz de caracteres que almacena la sopa de letras con la que operar.
2. Un entero positivo que determina el número de **F** filas de la matriz
3. Un entero positivo que determina el número de **C** columnas de la matriz.

Esta función imprime por pantalla la sopa de letras y no devuelve ningún resultado. Se asumirá que tanto F como C almacenan un valor mayor o igual que 1.

Para la siguiente sopa de letras de F filas y C columnas:

H	O	L	A	¶
	¶	M	E	S
	¶	¶	D	A
S	I	L	L	A
	¶			¶

Se imprimirá por pantalla lo siguiente:

```
H;O;L;A;¶
 ;¶;M;E;S
 ; ; ; ;
 ;¶;¶;D;A
S;I;L;L;A
 ;¶; ; ;¶
```

Nótese que se utiliza el carácter ";" como separador entre caracteres y un salto de línea para indicar el final de cada fila.

`search_puzzle`

Esta función determina si una palabra se puede insertar en la sopa de letras de forma horizontal a partir de una determinada coordenada de forma completa. Acepta los siguientes argumentos:

1. Dirección de una matriz de caracteres que almacena la sopa de letras.
2. Un entero positivo con el número **F** de filas de la matriz.
3. Un entero positivo con el número **C** de columnas de la matriz.
4. La dirección de una cadena de caracteres que almacena la palabra a insertar.
5. Un número entero mayor o igual a 0 con la coordenada **f** de la fila.
6. Un número entero mayor o igual a 0 con la coordinada **c** de la columna.

La función determinará si a partir de la coordenada (f,c) se puede insertar la palabra (cuarto argumento) de forma horizontal en la sopa de letras de forma completa, con independencia de lo que haya antes de esa coordenada.

Solo se tendrán en cuenta como caracteres de relleno al código ASCII 182. En caso de que haya algún carácter ya insertado se asumirá como un espacio en blanco. La función devuelve 0 si no se puede insertar y 1 si se puede insertar de forma completa y exacta. En caso de que la coordenada pasada se encuentre el código de relleno (código ASCIII 182, ¶) siempre se devolverá 0.

Por ejemplo:

- Para la sopa de letras anterior, la palabra "casa" y la coordenada (4,0) la función devolverá 0, porque a partir de esa coordenada hay 5 espacios y "casa" solo 4. Por tanto, no cabe de forma exacta en el espacio existente.

- Para la sopa de letras anterior, la palabra “casas” y la coordenada (4,0) la función devolverá 1, porque hay 5 espacios libres y “casas” tiene 5 y cabe de forma completa en el espacio existente.
- Para la sopa de letras anterior, la palabra “personas” y la coordenada (4,0) la función devolverá 0, porque a partir de la coordenada (4,0) solo hay 5 espacios libres y “personas” tiene 8 caracteres y no cabe de forma completa en el espacio libre.
- Para la sopa de letras anterior, la palabra “mas” y la coordenada (4,2), la función devolverá 1, porque a partir de la coordenada (4,2) quedan tres espacios hasta el final de la columna y “mas” tiene tres caracteres.
- Para la sopa de letras anterior, la palabra “es” y la coordenada (4,2), la función devolverá 0, porque a partir de la coordenada (4,2) quedan tres espacios hasta el final de la columna y “es” tiene solo dos caracteres.

`insert_puzzle`

Esta función inserta una palabra en una sopa de letras de forma horizontal a partir de una determinada coordenada. Acepta los siguientes argumentos:

1. Dirección de una matriz de caracteres que almacena la sopa de letras.
2. Un entero positivo con el número de filas de la matriz
3. Un entero positivo con el número de columnas de la matriz.
4. La dirección de una cadena de caracteres que almacena la palabra a insertar.
5. Un número entero mayor o igual a 0 con la coordenada f de la fila.
6. Un número entero mayor o igual a 0 con la coordinada c de la columna.

La función devuelve 0 si la palabra no se puede insertar y 1 en caso de que se pueda insertar de forma completa y exacta (si sobra espacio al final devuelve 0). En este caso, la función insertará la palabra (cuarto argumento) a partir de la coordenada (f,c).

Esta función, por tanto, comprobará en primer lugar que la palabra cabe en el espacio indicado utilizando la función **`search_puzzle`**. Si la palabra no cabe o no cabe de forma exacta, la función devolverá 0 y no se insertará nada en la sopa de letras.

`fill_puzzle`

Esta función crea una sopa de letras a partir de una lista de coordenadas. Esta lista estará formada por un vector V que almacenará las coordenadas de la matriz donde hay que insertar símbolo ¶ (código ASCII 182).

De esta forma:
el par (V[i], V[i+1]) para i = 0, 2, 4,…. Determina la coordenada donde ha de insertarse el símbolo ¶ (código ASCII 182).

La función acepta los siguientes argumentos:

1. Dirección de una matriz de caracteres que almacena la sopa de letras en la que se quiere insertar los símbolos de relleno.
2. número entero mayor o igual a 0 con el número de filas de la matriz.
3. número entero mayor o igual a 0 con el número de columnas de la matriz.
4. La dirección de un vector V que almacena las coordenadas donde insertar el símbolo ¶.
5. Un entero que determina el número de elementos del vector. Para un vector que almacene 5 coordenadas, se pasará el valor 10.

La función devuelve:

- El valor 1 en caso de que se pueda rellenar correctamente.
- El valor 0 en caso de que alguna de las coordenadas del vector V se encuentren fuera de rango. Para simplificar el proceso, se irán rellenando las coordenadas que estén en rango y en cuanto se encuentre una que no lo está se devolverá -1 y no se rellenará nada más. No será necesario eliminar los símbolos ¶ que ya se hayan insertado hasta el momento.

Para la sopa de letras:

				¶
	¶			
	¶	¶		

El vector de coordenadas que permite rellenarla es: { (0, 4), (1,1), (3,1), (3,2) } y el número de elementos del vector es 8 (4 coordenadas x 2).

Una posible forma de definir este vector en el segmento de datos sería la siguiente:

```
.data

     .align 2

     V: .word 0, 4, 1, 1, 3, 1, 3, 2
```

Consideraciones a tener en cuenta

En la realización de los ejercicios anteriores habrá de utilizarse **obligatoriamente** la siguiente función de biblioteca que se proporciona como material de apoyo:

`length`

Esta función acepta un argumento que representa una cadena de caracteres y la función devuelve la longitud de la cadena, es decir, el número de caracteres que tiene.
Por ejemplo, la longitud de “Hola” es 4.

Requisitos a tener en cuenta en la realización de los ejercicios

Se valorará la presentación de pseudocódigos de las funciones solicitadas de forma que se describa claramente el algoritmo implementado en ensamblador RISC-V.

Pueden desarrollarse funciones auxiliares adicionales si se considera necesario.

Para el desarrollo de todas las funciones ha de seguirse obligatoria y correctamente el convenio de paso de parámetros a funciones y uso de registros descrito en la asignatura.

Solo la función **`puzzle_print`** imprimirá por la pantalla, el resto NO pueden imprimir nada por pantalla.

Las funciones desarrolladas no leerán ningún por teclado.

Las funciones que entregue han de funcionar **sin** depender de un segmento de datos específico. El código enviado no incluiría ningún segmento de datos ni la función **`main`**, solo las funciones indicadas anteriormente y cualquier otra función auxiliar que haya sido necesario desarrollar.

Aspectos importantes a tener en cuenta

Normas generales

1) Los nombres de las funciones indicadas en el enunciado han de escribirse **obligatoriamente en la forma indicada, tal y como están escritas**.
2) Las prácticas se realizarán en **grupos de dos estudiantes**.
3) La entrega de la práctica se realizará a través de los entregadores habilitados. **No** se permite la entrega a través de correo electrónico.
4) La entrega se realizará en el plazo dado por los entregadores.
5) Se prestará especial atención a detectar funcionalidades copiadas entre dos o más prácticas. En caso de encontrar implementaciones comunes en dos prácticas (o contenidos similares en la memoria), ambas obtendrán una calificación de 0 (cero).
6) Debe seguirse el convenio de paso de parámetros descrito en clase. **Aquellas funciones que no sigan correctamente el convenio de paso de parámetros y uso de registros descritos en la asignatura también serán calificadas con un 0 (cero).**
7) Han de entregarse todos los ejercicios solicitados. Si un ejercicio no se entrega, se considerará la práctica como **no entregada**.
8) Los ejercicios que no compilen o que no se ajusten a la funcionalidad y requisitos planteados, obtendrán una calificación de 0 (cero).
9) Un programa no comentado obtendrá una calificación de 0 (cero).
10) **El uso de IA Generativa para ayudar o completar cualquier parte de esta práctica será penalizado académicamente. En caso de detectarse su uso en el desarrollo la práctica, la práctica será calificada con un 0.**

Memoria de la práctica

1) La memoria (un único documento) tendrá que contener al menos los siguientes apartados:
    - Portada donde figuren los autores (incluyendo nombre completo, NIA, dirección de correo electrónico, grupo de clase al que pertenecen y titulación cursada).
    - Índice de contenidos.
    - Contenidos pedidos en los distintos ejercicios (una sección por ejercicio).
    - Conclusiones, problemas encontrados y estimación del tiempo empleado en la práctica.
2) **La longitud de la memoria no deberá superar las 10 páginas** (portada e índice incluidos).
3) Al respecto de la posible descripción de los programas pedidos:
    - La memoria debe describir el comportamiento de los programas, así como las principales decisiones de diseño. **Se deberá incluir el pseudocódigo correspondiente a cada uno de los ejercicios de esta práctica.**
    - Se ha de incluir la **batería de pruebas (como la definida en la siguiente sección)** utilizadas para validar la funcionalidad de las funciones solicitadas y los **resultados** obtenidos. Se dará mayor puntuación a pruebas avanzadas, casos extremos, y en general a aquellas pruebas que garanticen el correcto funcionamiento de la práctica en todos los casos.
        - Evite pruebas duplicadas que evalúan los mismos flujos de programa. La puntuación de este apartado no se mide en función del número de pruebas, sino del grado de cobertura de las mismas. Es mejor pocas pruebas que evalúan diferentes casos a muchas pruebas que evalúan siempre el mismo caso.

<u>NOTA:</u> NO DESCUIDE LA CALIDAD DE LA MEMORIA DE SU PRÁCTICA.

Aprobar la memoria es tan imprescindible para aprobar la práctica, como el correcto funcionamiento de esta. Si al evaluarse la memoria de su práctica, se considera que no alcanza el mínimo admisible, su práctica estará suspensa.

Definición de pruebas

Para la sección de definición y batería de pruebas de cada uno de los ejercicios que se debe incluir en la memoria una tabla que seguirá el siguiente formato:

Datos a introducir:	**Descripción de la prueba:**	**Resultado esperado:**	**Resultado obtenido:**

Procedimiento de entrega de la práctica.

La entrega de la práctica 1 se realizará de forma electrónica a través de Aula Global.

La práctica se hará en grupos de **dos estudiantes**.

La fecha límite de entrega para la entrega es el día **1 de Noviembre de 2026, a las 23:55 horas.**

Es posible entregar tantas veces como quiera dentro del plazo dado, la única versión registrada de su práctica es la última entregada. La valoración de la práctica es la valoración del contenido de esta última entrega. Recuerde revisar siempre lo que entregue.

Entregador: Se deberá entregar un único archivo comprimido en formato **zip** con el nombre ec_p1_AAAAAAAAA_BBBBBBBBB.zip donde AAAAAAAAA y BBBBBBBBB son los NIA de los integrantes del grupo.

El archivo **zip** debe contener solo los siguientes archivos:

- **exercises.s**
  Este archivo solo contendrá el código todas las funciones desarrolladas y de todas las funciones auxiliares que se necesiten. **No tendrá ni función *main* ni segmento de datos.** Por tanto, las funciones que entregue han de funcionar **sin** depender de un segmento de datos específico.
- **report.pdf**
  Este archivo contendrá el contenido de la memoria, y tendrá formato PDF (no vale renombrar un fichero de texto o similar con extensión .pdf).
- **authors.txt**
  Este archivo de texto contendrá una línea para cada integrante del grupo con el NIA y el grupo al que pertenece separados por un punto y coma. Ejemplo: 1000001;81

Evaluación de la práctica

- Para la evaluación de la práctica se utilizará el siguiente baremo:
    - Memoria: 10%.
    - Plan de pruebas: 20%
    - Código entregado: 70%

Tenga en cuenta que, **para seguir el proceso de evaluación continua**, la **nota media de las** dos **prácticas** ha de ser al menos de **4 sobre 10**. **Han de entregarse todas las funciones solicitadas** para considerarse que la práctica se ha entregado.

NOTAS:

1. **Si se detecta un error de concepto grave en la práctica (en cualquier apartado de cualquier ejercicio), la valoración global de toda la práctica será de cero puntos (0 puntos).**
2. **Si no respeta el formato de entrega (por ejemplo, no respetar el nombre de los ficheros pedidos, entregar un archivo .rar, entregar los ficheros en un directorio, etc.) la nota quedará reducida significativamente.**
3. **En caso de encontrar implementaciones comunes en dos prácticas (o contenidos similares en la memoria) se entenderá que se ha copiado la práctica y ambas obtendrán una calificación de 0.**
4. **En caso de encontrarse fragmentos de código obtenidos directamente de Internet se entenderá que se ha copiado dicho contenido y la práctica tendrá una calificación de 0.**